# Spring Boot — Lecture 7

Spring Bean Lifecycle — From Creation to Destruction

## 1. Overview — Why Bean Lifecycle Matters

The Bean Lifecycle describes every phase a Spring-managed object goes through — from the moment the IoC Container starts, to when the bean is destroyed. This is one of the most important interview topics in Spring.

| Who manages? | Lifecycle covers | Why it matters |
|---|---|---|
| Spring IoC Container | Bean creation → dependency injection → initialization → active use → destruction | Understand internals, write initialization/cleanup logic, debug issues, ace interviews |

*This lecture focuses on Singleton beans with Eager initialization — the default. Differences for Lazy Singleton and Prototype beans are covered at the end.*

## 2. Complete Bean Lifecycle — Step by Step

| Step | Phase | What happens |
|---|---|---|
| 0 | IoC Container starts | new AnnotationConfigApplicationContext(AppConfig.class) is called. Container boots up. |
| 1 | Read Configuration | Spring reads AppConfig via Reflection. Finds @Configuration, @ComponentScan, @Bean methods. |
| 2 | Component Scan | Spring scans the declared package recursively for all @Component-annotated classes. |
| 3 | Read Bean Definitions | For EVERY discovered class, Spring creates a BeanDefinition (metadata: name, scope, lazy, dependencies). All definitions created BEFORE any object is instantiated. |
| 4 | Instantiate Objects | Spring creates actual objects. Dependency-aware — creates dependencies first (e.g. PaymentService before OrderService if OrderService depends on it). |
| 5 | Inject Dependencies | Constructor injection: happens during step 4 (same step). Field/setter injection: happens after object is created. |
| 6 | Aware Interfaces called | If bean implements BeanNameAware or ApplicationContextAware, Spring calls setBeanName() / setApplicationContext() as callback methods. |
| 7 | Initialization Callbacks | @PostConstruct / afterPropertiesSet() / init-method called. Use for caching, resource loading, map initialization etc. |
| 8 | Bean is Ready | Bean is fully initialized and available for use. Application logic runs here. |
| 9 | Destruction Callbacks | context.close() called → @PreDestroy / destroy() / destroy-method called before removal. |
| 10 | Bean Destroyed | Bean removed from IoC Container. For singleton: container manages this. For prototype: garbage collection handles it. |

*AppConfig itself is also a Spring-managed bean — @Configuration internally uses @Component. So AppConfig has its own lifecycle too.*

## 3. BeanDefinition — Metadata Before Object Creation

Before creating any object, Spring creates a BeanDefinition for each bean. This metadata map tells Spring how to create and manage each bean.

| BeanDefinition field | Example value for OrderService |
|---|---|
| Bean name | orderService (class name, first letter lowercase) |
| Bean class | in.coderarmyarmy.OrderService |

| BeanDefinition field | Example value for OrderService |
|---|---|
| Scope | singleton (default) |
| Lazy | false (default for singleton) |
| Dependencies | paymentService (from constructor parameter) |
| Init method | null (unless specified) |
| Destroy method | null (unless specified) |

*ALL bean definitions are created before any object is instantiated. This is why Spring can determine initialization order correctly.*

## 4. Aware Interfaces — Callback Before Initialization

Aware interfaces let a bean receive information about its own Spring context. Spring calls these as callback methods automatically — you never call them yourself.

| Interface | Method Spring calls | What you receive |
|---|---|---|
| BeanNameAware | setBeanName(String name) | The name of this bean in the IoC Container |
| ApplicationContextAware | setApplicationContext(ApplicationContext ctx) | Reference to the ApplicationContext (IoC Container) this bean lives in |

### Example — implementing BeanNameAware:

```
@Component("userBean")  // custom bean name
public class UserService implements BeanNameAware, ApplicationContextAware {

    public UserService() {
        System.out.println("UserService constructor called");
    }

    // Spring calls this AUTOMATICALLY after object creation + DI
    @Override
    public void setBeanName(String name) {
        System.out.println("Bean name is: " + name);  // prints "userBean"
    }

    // Spring calls this AUTOMATICALLY — tells us which container this bean is in
    @Override
    public void setApplicationContext(ApplicationContext ctx) throws BeansException {
        System.out.println("ApplicationContext: " + ctx.getClass().getName());
    }
}

// Console output order:
// 1. UserService constructor called
// 2. Bean name is: userBean
// 3. ApplicationContext: ...AnnotationConfigApplicationContext
```

*Important: calling setBeanName() yourself does NOT change the bean's name in the IoC Container. These are read-only callbacks — Spring provides the information, you cannot change it.*

### Real-world use cases for Aware interfaces:

- Logging: prefix every log entry with the bean name and container type for easier debugging.
- Multi-container apps: two ApplicationContexts exist (XML + annotation) — use ApplicationContextAware to know which container a bean came from.
- 99% of business logic does NOT need these — they are infrastructure-level tools.

## 5. Initialization Callbacks — Run Code Before First Use

After the object is created and dependencies are injected, Spring gives you a hook to run initialization logic before any business method is called.

### Three approaches (use @PostConstruct — it's the modern standard):

| Approach | How to use | When to use |
|---|---|---|
| @PostConstruct (recommended) | Add @PostConstruct on any method — Spring calls it after DI is complete | All new code. Requires jakarta.annotation dependency in plain Spring Core. |
| InitializingBean interface (legacy) | Implement InitializingBean, override afterPropertiesSet() | Old codebases. Tightly couples your class to Spring API. |
| init-method attribute on @Bean | @Bean(initMethod="start") — Spring calls start() on the returned object | When creating bean via @Bean in AppConfig (for third-party classes). |

### @PostConstruct example:

```
@Component
public class CartService {
    private Map cart;

    public CartService() {
        this.cart = new HashMap<>();
        System.out.println("CartService constructor called");
        // DO NOT do heavy work here — keep constructors light!
    }

    @PostConstruct  // Spring calls this AFTER constructor + DI — before any business method
    public void init() {
        // Safe to use injected dependencies here (field/setter injection is complete)
        cart.put(1, "Aashutosh");
        cart.put(2, "Rohit");
        System.out.println("Bean is ready");
        // Good for: cache warming, resource loading, map initialization
    }

    public String getValue(int key) {
        return cart.get(key);
    }
}

// Console output:
// CartService constructor called
// Bean is ready

// Then in Main:
// context.getBean(CartService.class).getValue(1)  →  "Aashutosh"
```

### InitializingBean (legacy) example:

```
@Component
public class CartService implements InitializingBean {

    @Override
    public void afterPropertiesSet() throws Exception {
        // Called by Spring after DI complete — same timing as @PostConstruct
        cart.put(1, "Aashutosh");
        System.out.println("Bean is ready");
    }
}
```

### init-method via @Bean (for third-party or complex classes):

```
// In AppConfig.java
@Bean(initMethod = "start", destroyMethod = "stop")
public CartService getCartBean() {
    return new CartService();
}

// In CartService.java — no annotations needed
public void start() {
    cart.put(1, "Aashutosh");
    System.out.println("Bean is ready");
}
```

## 6. Why @PostConstruct — Not Constructor?

This is a common interview question. The key difference is timing.

| Constructor | @PostConstruct |
|---|---|
| Called when object is being created | Called after object is created AND dependencies are injected |
| Field/setter dependencies are NOT yet available | All dependencies (field, setter, constructor) are guaranteed to be available |
| Should be kept lightweight — just initialize basic state | Safe to use injected beans, perform cache warming, load resources |
| Heavy operations slow down object creation | Heavy operations done after clean, fast object construction |

```
// WHY you need @PostConstruct (not constructor):
@Component
public class OrderService {

    @Autowired  // field injection — NOT available in constructor yet!
    private PaymentService paymentService;

    public OrderService() {
        // paymentService is NULL here — injected AFTER constructor
        // paymentService.somMethod()  ← NullPointerException!
    }

    @PostConstruct
    public void init() {
        // paymentService is fully injected NOW — safe to use
        paymentService.someMethod();  // works perfectly
    }
}
```

## 7. Destruction Callbacks — Cleanup Before Bean is Removed

Mirror of initialization callbacks. Called when context.close() is invoked or when the application shuts down. Use for cleanup: closing connections, clearing caches, releasing resources.

| Approach | How to use | When to use |
|---|---|---|
| @PreDestroy (recommended) | Add @PreDestroy on any method — Spring calls it before destroying the bean | All new code. Symmetric to @PostConstruct. |
| DisposableBean interface (legacy) | Implement DisposableBean, override destroy() | Old codebases. Tightly couples class to Spring API. |
| destroyMethod attribute on @Bean | @Bean(destroyMethod="stop") — Spring calls stop() before removing the bean | When creating bean via @Bean in AppConfig. |

### @PreDestroy example:

```
@Component
public class CartService {
    private Map cart = new HashMap<>();

    @PostConstruct
    public void init() {
        cart.put(1, "Aashutosh");
        System.out.println("Bean is ready");
    }

    @PreDestroy  // Spring calls this BEFORE destroying the bean
    public void cleanup() {
        cart.clear();  // release resources
        System.out.println("Bean is getting destroyed");
    }
}

// How to trigger destruction — close the context:
ConfigurableApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);

CartService cart = context.getBean(CartService.class);
System.out.println(cart.getValue(1));  // Aashutosh

context.close();  // → triggers @PreDestroy callback
// Console: "Bean is getting destroyed"
```

*Use ConfigurableApplicationContext instead of ApplicationContext to access context.close().*
*ConfigurableApplicationContext extends ApplicationContext and adds the close() method.*

## 8. All Callbacks Together — Console Output Order

```
@Component("userBean")
public class CartService
    implements BeanNameAware, ApplicationContextAware, InitializingBean, DisposableBean {

    private Map cart = new HashMap<>();

    // STEP 4: Object created
    public CartService() {
        System.out.println("1. Constructor called");
    }

    // STEP 5: DI happens (field/setter injection after constructor)

    // STEP 6: Aware interfaces
    @Override
    public void setBeanName(String name) {
        System.out.println("2. Bean name: " + name);
    }

    @Override
    public void setApplicationContext(ApplicationContext ctx) throws BeansException {
        System.out.println("3. ApplicationContext: " + ctx.getClass().getSimpleName());
    }

    // STEP 7: Initialization callback
    @PostConstruct                      // OR afterPropertiesSet() — same timing
    public void init() {
        cart.put(1, "Aashutosh");
        System.out.println("4. Bean is ready (@PostConstruct)");
    }
```

```
    // STEP 8: Business methods (called from Main)
    public String getValue(int key) { return cart.get(key); }

    // STEP 9: Destruction callback (on context.close())
    @PreDestroy                          // OR destroy() — same timing
    public void cleanup() {
        cart.clear();
        System.out.println("5. Bean is getting destroyed");
    }
}

// Console output:
// 1. Constructor called
// 2. Bean name: userBean
// 3. ApplicationContext: AnnotationConfigApplicationContext
// 4. Bean is ready (@PostConstruct)
//    (application runs here — getValue() etc.)
// 5. Bean is getting destroyed  ← after context.close()
```

## 9. Lifecycle Variations — Lazy Singleton vs Prototype

| Phase | Singleton (Eager, default) | Singleton (Lazy, @Lazy) | Prototype (@Scope) |
|---|---|---|---|
| Steps 0–3 (container start → bean definitions) | All happen at startup | All happen at startup | All happen at startup |
| Steps 4–7 (instantiate → init callbacks) | AT STARTUP — before any request | On FIRST REQUEST — when getBean() or DI triggers it | On EVERY REQUEST — new object each time |
| Step 8 (active use) | Same object returned forever | Same object returned after first creation | New object for every getBean() call |
| Step 9 (@PreDestroy called?) | YES — when context.close() | YES — when context.close() | NO — Spring does not manage destruction |
| Step 10 (destroyed by) | Spring IoC Container on close() | Spring IoC Container on close() | Java Garbage Collector — when no references remain |

*Prototype beans: Spring creates them and hands them over to the caller. After that, Spring has NO reference to them — they are garbage collected normally. This prevents memory leaks from accumulating objects in the container.*

## 10. Using @PostConstruct to Resolve Circular Dependency

If you have a circular dependency (A needs B, B needs A), @PostConstruct can be used as a last resort. But remember: refactoring the design is always the right fix.

```
@Component
public class A {
    private B b;

    // NO @Autowired on B — constructor injection would fail (circular dep)
    public A(B b) { this.b = b; }

    // BUT B needs A — we inject A into B manually after both are created
}

@Component
public class B {
    private A a;

    // Empty constructor — no dependency on A at creation time
    public B() {}
```

```
    // Setter for A (no @Autowired — we call this manually in @PostConstruct)
    public void setA(A a) { this.a = a; }
}

// In A:
@PostConstruct
public void wireB() {
    // At this point: A is created, B is created, A.b is injected
    // B.a is still null — we inject it now
    b.setA(this);  // pass A's reference into B
}
// Now both A.b and B.a are set — circular dependency resolved
// (Still a hack — refactor to break the circle instead!)
```

## 11. Quick Reference — All Lifecycle Annotations & Interfaces

| Annotation / Interface | Phase | Recommended? | Notes |
|---|---|---|---|
| @PostConstruct | Initialization callback | YES — use this | Requires jakarta.annotation. Built into Spring Boot. |
| InitializingBean.afterPropertiesSet() | Initialization callback | Legacy only | Tightly couples class to Spring API. |
| @Bean(initMethod="start") | Initialization callback | YES for @Bean | Use when creating via @Bean in AppConfig. |
| @PreDestroy | Destruction callback | YES — use this | Mirror of @PostConstruct. Requires jakarta.annotation. |
| DisposableBean.destroy() | Destruction callback | Legacy only | Tightly couples class to Spring API. |
| @Bean(destroyMethod="stop") | Destruction callback | YES for @Bean | Mirror of initMethod for AppConfig-managed beans. |
| BeanNameAware.setBeanName() | Aware interface | Rare use | Get bean name for logging. Read-only — can't change the name. |
| ApplicationContextAware.setApplicationContext() | Aware interface | Rare use | Identify which container the bean lives in. |

### Dependency for @PostConstruct / @PreDestroy in plain Spring Core:

```
    jakarta.annotation
    jakarta.annotation-api
    3.0.0
```

---

*Next Lecture: XML-based Configuration (legacy approach) — , ,*